

ASSIGNMENT NO. 2

Question 1 — Inventory Management System

You are hired as a junior C++ developer at a retail company. The warehouse team manages hundreds of product items manually on paper. Your manager asks you to write a small inventory module.

"We need to store product records, look them up, calculate discounts, and print summaries. Also, our existing

codebase has separate add functions for int and double quantities — clean that up using function overloading."

Part A — Product class with Array of Objects

Design a class Product with the following private data members:

- int productId
- string name
- double price
- int quantity

Public member functions:

void acceptDetails(); // Read all fields from user

void displayDetails() const; // Print formatted product info

double totalValue() const; // price * quantity

bool isLowStock(int threshold) const; // true if quantity < threshold

In main():

1. Create an array of 5 Product objects on the stack
2. Accept details for all 5 using a loop
3. Display all products in a formatted table
4. Print the product with the highest total value
5. Print all products where **quantity is below a threshold entered by the user**

Code:

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;
class Product {
private:
    int productId;
    string name;
    double price;
    int quantity;
```

```

public:
    void acceptDetails() {
        cout << "\nEnter Product ID: ";
        cin >> productId;

        cout << "Enter Product Name: ";
        cin >> name;

        cout << "Enter Price: ";
        cin >> price;

        cout << "Enter Quantity: ";
        cin >> quantity;
    }

    void displayDetails() const {
        cout << left << setw(6) << productId
            << left << setw(12) << name
            << left << setw(10) << fixed << setprecision(2) << price
            << left << setw(8) << quantity
            << left << setw(10) << fixed << setprecision(2) << totalValue();

        if (isLowStock(10)) {
            cout << " <- LOW STOCK";
        }
        cout << endl;
    }

    double totalValue() const {
        return price * quantity;
    }

    bool isLowStock(int threshold) const {
        return quantity < threshold;
    }

    int getId() const { return productId; }
    string getName() const { return name; }
    int getQuantity() const { return quantity; }
};

int main() {
    const int SIZE = 5;

```

```

Product products[SIZE];

cout << "===== ENTER PRODUCT DETAILS =====";
for (int i = 0; i < SIZE; i++) {
    cout << "\n--- Product " << (i + 1) << " ---";
    products[i].acceptDetails();
}

cout << "\n\n===== INVENTORY REPORT =====\n";
cout << left << setw(6) << "ID"
    << left << setw(12) << "Name"
    << left << setw(10) << "Price"
    << left << setw(8) << "Qty"
    << left << setw(10) << "Total Value" << endl;

for (int i = 0; i < SIZE; i++) {
    products[i].displayDetails();
}

int highestIndex = 0;
for (int i = 1; i < SIZE; i++) {
    if (products[i].totalValue() > products[highestIndex].totalValue()) {
        highestIndex = i;
    }
}
cout << "\nHighest Value Product: " << products[highestIndex].getName()
    << " (Rs. " << fixed << setprecision(2)
    << products[highestIndex].totalValue() << ")" << endl;

int threshold;
cout << "\nEnter quantity threshold to check low stock: ";
cin >> threshold;

cout << "\nLow Stock Products (Qty < " << threshold << "):" << endl;
bool foundLowStock = false;
for (int i = 0; i < SIZE; i++) {
    if (products[i].isLowStock(threshold)) {
        cout << "- " << products[i].getName()
            << " (Qty: " << products[i].getQuantity() << ")" << endl;
        foundLowStock = true;
    }
}
if (!foundLowStock) {
    cout << "No products are below the threshold." << endl;
}

```

```
}  
  
    return 0;  
}
```

Expected Output :

===== INVENTORY REPORT =====

ID	Name	Price	Qty	Total Value
1001	Widget-A	250.00	120	30000.00
1002	Bolt-B	15.50	8	124.00

<- LOW STOCK

Highest Value Product: Widget-A (Rs. 30000.00)

Enter quantity threshold to check low stock: 15

Low Stock Products (Qty < 15):

- Bolt-B (Qty: 8)

Part B — Function Overloading

The warehouse system needs to calculate restocking costs for different quantity types. Write the following overloaded functions (not inside any class):

```
double reorderCost(int qty, double unitPrice);  
double reorderCost(double qty, double unitPrice);  
double reorderCost(int qty, double unitPrice, double taxRate);
```

Also write a function with a default argument:

```
double applyDiscount(double price, double discountPercent = 10.0);
```

Call all versions from main() and print the results.

Code:

```
#include <iostream>  
#include <iomanip>  
using namespace std;
```

```

double reorderCost(int qty, double unitPrice) {
    return qty * unitPrice;
}

double reorderCost(double qty, double unitPrice) {
    return qty * unitPrice;
}

double reorderCost(int qty, double unitPrice, double taxRate) {
    double baseCost = qty * unitPrice;
    double tax = baseCost * (taxRate / 100.0);
    return baseCost + tax;
}

double applyDiscount(double price, double discountPercent = 10.0) {
    double discountAmount = price * (discountPercent / 100.0);
    return price - discountAmount;
}

int main() {
    cout << fixed << setprecision(2);
    int qty1 = 50;
    double price1 = 20.0;
    cout << "Reorder Cost (int qty, no tax): Rs. "
        << reorderCost(qty1, price1) << endl;

    double qty2 = 12.5;
    double price2 = 40.0;
    cout << "Reorder Cost (double qty, no tax): Rs. "
        << reorderCost(qty2, price2) << endl;

    int qty3 = 30;
    double price3 = 15.0;
    double tax3 = 18.0;
    cout << "Reorder Cost (int qty, with " << tax3 << "% tax): Rs. "
        << reorderCost(qty3, price3, tax3) << endl;

    double originalPrice = 500.0;

    cout << "\nPrice after default discount (10%): Rs. "
        << applyDiscount(originalPrice) << endl;

    cout << "Price after custom discount (25%): Rs. "

```

```
<< applyDiscount(originalPrice, 25.0) << endl;

return 0;
}
```

Output:

Reorder Cost (int qty, no tax): Rs. 1000.00
Reorder Cost (double qty, no tax): Rs. 500.00
Reorder Cost (int qty, with 18.00% tax): Rs. 531.00

Price after default discount (10%): Rs. 450.00
Price after custom discount (25%): Rs. 375.00

Question 2 — Hospital Patient Registry

A government hospital is replacing its paper-based patient records system. You are on the team building the Patient module. The system needs to handle patients who walk in with full details, patients registered in emergencies (name and ID only), and bulk ward admission.

"We need proper constructor overloading so the system handles all these cases without separate functions. And since patient records are created and destroyed throughout the day, we need to see the object lifecycle clearly."

Design a class Patient with:

Private data members:

- int patientId
- string name
- int age
- string ward
- const string bloodGroup ← must be set at construction time only

Public member functions:

```
// Constructor 1 — Default: id=0, name="Unknown", age=0, ward="General",
bloodGroup="O+"
**Patient();
// Constructor 2 — Emergency admission: only id and name known Patient(int id, const string& name);**

// Constructor 3 — Full admission details
Patient(int id, const string& name, int age, const string& ward, const string& bg);

// Destructor — print "Patient <name> discharged." when object is destroyed
~Patient();

void displayRecord() const;
```

```
void transferWard(const string& newWard);
```

Use Constructor Initializer List to initialise bloodGroup in all three constructors (it is const — it cannot be assigned inside the body).

In main():

1. Create one patient of each type (3 stack objects) — observe constructor messages
2. Create a dynamic array of 4 patients on the heap using new Patient[4] (default constructor)
3. Use a loop to call displayRecord() on all 4
4. Transfer one patient's ward using transferWard()
5. delete[] the array and observe destructor messages
6. Let the 3 stack objects go out of scope and observe destructor messages

Code:

```
#include <iostream>
#include <string>
using namespace std;
class Patient {
private:
    int patientId;
    string name;
    int age;
    string ward;
    const string bloodGroup;

public:
    Patient()
        : patientId(0), name("Unknown"), age(0),
          ward("General"), bloodGroup("O+") {
        cout << "[Constructor] Default patient registered." << endl;
    }

    Patient(int id, const string& name)
        : patientId(id), name(name), age(0),
          ward("General"), bloodGroup("O+") {
        cout << "[Constructor] Emergency: " << name << endl;
    }

    Patient(int id, const string& name, int age,
            const string& ward, const string& bg)
        : patientId(id), name(name), age(age),
          ward(ward), bloodGroup(bg) {
        cout << "[Constructor] Full admission: " << name << endl;
    }
}
```

```

~Patient() {
    cout << "[Destructor] Patient " << name
        << " discharged." << endl;
}

void displayRecord() const {
    cout << "Patient Record:" << endl;
    cout << "ID      : " << patientId << endl;
    cout << "Name    : " << name << endl;
    cout << "Age     : " << age << endl;
    cout << "Ward    : " << ward << endl;
    cout << "Blood Grp : " << bloodGroup << endl;
    cout << endl;
}

void transferWard(const string& newWard) {
    cout << "Ward Transfer: " << name
        << " -> " << newWard << endl;
    ward = newWard;
}
};

int main() {
    Patient p1(1001, "Meera Joshi", 34, "Cardiology", "B+");
    Patient p2(1002, "Raj Patel");
    Patient p3;

    cout << endl;
    p1.displayRecord();
    p2.transferWard("ICU");

    cout << endl;
    Patient* patients = new Patient[4];

    cout << endl;
    for (int i = 0; i < 4; i++) {
        patients[i].displayRecord();
    }
    cout << "--- Deleting Dynamic Array ---" << endl;
    delete[] patients;

    return 0;
}

```


Expected Output:

[Constructor] Full admission: Meera Joshi

[Constructor] Emergency: Raj Patel

[Constructor] Default patient registered.

Patient Record:

ID : 1001

Name : Meera Joshi

Age : 34

Ward : Cardiology

Blood Grp : B+

Ward Transfer: Raj Patel -> ICU

[Destructor] Patient Meera Joshi discharged.

[Destructor] Patient Raj Patel discharged.

Question 3 — Game Engine Entity System

You are interning at a game studio. The engine team asks you to implement a basic Entity class for game objects (players, enemies, items). The lead developer sends you this note:

"Use this pointer in setters so we can chain calls —

entity.setName("Archer").setHealth(100).setLevel(5) — this is standard in our codebase. Also, we're moving some math utilities into their own namespace so they don't collide with standard library names.

And the matrix (2D grid) for the game map needs dynamic allocation — the size isn't known at compile time."

Part A — this Pointer and Method Chaining

Design a class Entity with private data members:

- string name
- int health
- int level
- string type ("Player", "Enemy", "Item")

All setters must return Entity& so calls can be chained:

Entity& setName(const string& name); Entity& setHealth(int health);

Entity& setLevel(int level);

Entity& setType(const string& type);

Getters (all const):

```
string getName() const;
int getHealth() const;
int getLevel() const;
string getType() const;
void displayInfo() const;
```

In main(), create at least 3 entities using method chaining:

```
Entity player, enemy, item;
player.setName("Aragorn").setHealth(100).setLevel(10).setType("Player");
enemy.setName("Orc").setHealth(60).setLevel(5).setType("Enemy");
item.setName("HealthPotion").setHealth(0).setLevel(1).setType("Item");
```

Code:

```
#include <iostream>
#include <string>
using namespace std;
class Entity {
private:
    string name;
    int health;
    int level;
    string type;

public:
    Entity& setName(const string& name) {
        this->name = name;
        return *this;
    }

    Entity& setHealth(int health) {
        this->health = health;
        return *this;
    }

    Entity& setLevel(int level) {
        this->level = level;
        return *this;
    }

    Entity& setType(const string& type) {
        this->type = type;
        return *this;
    }
}
```

```

string getName() const {
    return name;
}

int getHealth() const {
    return health;
}

int getLevel() const {
    return level;
}

string getType() const {
    return type;
}

void displayInfo() const {
    cout << "Name   : " << name << endl;
    cout << "Health : " << health << endl;
    cout << "Level  : " << level << endl;
    cout << "Type   : " << type << endl;
    cout << endl;
}
};

int main() {
    Entity player, enemy, item;
    player.setName("Aragorn")
        .setHealth(100)
        .setLevel(10)
        .setType("Player");
    enemy.setName("Orc")
        .setHealth(60)
        .setLevel(5)
        .setType("Enemy");
    item.setName("HealthPotion")
        .setHealth(0)
        .setLevel(1)
        .setType("Item");
    cout << "--- Player ---" << endl;
    player.displayInfo();

    cout << "--- Enemy ---" << endl;

```

```
    enemy.displayInfo();

    cout << "--- Item ---" << endl;
    item.displayInfo();
    return 0;
}
```

Output:

```
--- Player ---
Name  : Aragorn
Health : 100
Level  : 10
Type   : Player
```

```
--- Enemy ---
Name  : Orc
Health : 60
Level  : 5
Type   : Enemy
```

```
--- Item ---
Name  : HealthPotion
Health : 0
Level  : 1
Type   : Item
```

Part B — Namespaces

The engine has two modules: Physics and GameMath. Both need a clamp() function, but for different purposes.

Define both namespaces with their own clamp() and lerp() functions:

```
namespace Physics {
double clamp(double val, double min, double max); // clamp a velocity
double lerp(double a, double b, double t);        // linear interpolation for physics
}
```

```
namespace GameMath {
int clamp(int val, int min, int max); (health, level)
double lerp(double a, double b, double t);
}
```

Call both versions from main() using Physics::clamp() and GameMath::clamp(). Also demonstrate using namespace in a limited block scope (not at the top of the file).

Code:

```
#include <iostream>
using namespace std;
namespace Physics {
    double clamp(double val, double min, double max) {

        if (val < min)
            return min;

        if (val > max)
            return max;

        return val;
    }
    double lerp(double a, double b, double t) {

        return a + (b - a) * t;
    }
}

namespace GameMath {
    int clamp(int val, int min, int max) {

        if (val < min)
            return min;

        if (val > max)
            return max;

        return val;
    }

    double lerp(double a, double b, double t) {
        return a + (b - a) * t;
    }
}

int main() {
    double velocity = Physics::clamp(150.5, 0.0, 100.0);
    cout << "Physics Clamp: " << velocity << endl;

    int health = GameMath::clamp(120, 0, 100);
```

```

cout << "GameMath Clamp: " << health << endl;

double physicsResult = Physics::lerp(10.0, 20.0, 0.5);
cout << "Physics Lerp: " << physicsResult << endl;

double mathResult = GameMath::lerp(100.0, 200.0, 0.25);
cout << "GameMath Lerp: " << mathResult << endl;

{
    using namespace Physics;
    double speed = clamp(120.0, 0.0, 100.0);

    cout << "Using Physics namespace: "
         << speed << endl;
}

int level = GameMath::clamp(15, 1, 10);
cout << "Level after clamp: "
     << level << endl;
return 0;
}

```

Output:

```

Physics Clamp: 100
GameMath Clamp: 100
Physics Lerp: 15
GameMath Lerp: 125
Using Physics namespace: 100
Level after clamp: 10

```

Part C — Dynamic 2D Game Map

The game map is an $R \times C$ grid of integers (each cell stores a tile type). R and C are read from the user at runtime — so the map must be allocated on the heap using `int**`.

Requirements:

1. Allocate the 2D array using `new int*[R]` and then `new int[C]` for each row
2. Fill the grid: accept values from user or use `rand() % 5` to simulate tile types (0=grass, 1=water, 2=mountain, 3=forest, 4=dungeon)
3. Display the grid with a legend
4. Find and print the count of each tile type
5. Deallocate properly: `delete[]` each row, then `delete[]` the outer array

Code:

```
#include <iostream>
using namespace std;

int main() {
    int R, C;
    cout << "Enter number of rows: ";
    cin >> R;

    cout << "Enter number of columns: ";
    cin >> C;

    int** map = new int*[R];
    for (int i = 0; i < R; i++) {
        map[i] = new int[C];
    }

    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            map[i][j] = rand() % 5;
        }
    }

    cout << "\n===== GAME MAP (" << R
        << " x " << C << ") =====" << endl;

    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            cout << " " << map[i][j];
        }
        cout << endl;
    }

    cout << "\nLegend: "
        << "0=Grass "
        << "1=Water "
        << "2=Mountain "
        << "3=Forest "
        << "4=Dungeon"
        << endl;

    int grass = 0;
    int water = 0;
    int mountain = 0;
```

```

int forest = 0;
int dungeon = 0;

for (int i = 0; i < R; i++) {
    for (int j = 0; j < C; j++) {
        if (map[i][j] == 0)
            grass++;

        else if (map[i][j] == 1)
            water++;

        else if (map[i][j] == 2)
            mountain++;

        else if (map[i][j] == 3)
            forest++;

        else if (map[i][j] == 4)
            dungeon++;
    }
}

cout << "\nTile Count:" << endl;

cout << "Grass    : " << grass << endl;
cout << "Water     : " << water << endl;
cout << "Mountain  : " << mountain << endl;
cout << "Forest    : " << forest << endl;
cout << "Dungeon   : " << dungeon << endl;

for (int i = 0; i < R; i++) {
    delete[] map[i];
}
delete[] map;
return 0;
}

```

Expected Output :

Enter number of rows: 3

Enter number of columns: 4

===== **GAME MAP (3 x 4)** =====

3 1 2 0

3 0 1 2

4 1 2 2

Legend: 0=Grass 1=Water 2=Mountain 3=Forest 4=Dungeon

Tile Count:

Grass : 2

Water : 3

Mountain : 4

Forest : 2

Dungeon : 1